

APPM 4600 HW2 - Charlie Caravati

```
In [18]: import numpy as np
import math
import matplotlib.pyplot as plt
```

1

(a)

To show that  $x \sin \sqrt{x} = O(x^{3/2})$  as  $x \rightarrow 0$ , we show  $\lim_{x \rightarrow 0} \frac{f(x)}{g(x)} = M < \infty$  for some positive constant  $M$ .

Letting  $f(x) = x \sin \sqrt{x}$  and  $g(x) = x^{3/2}$

$\lim_{x \rightarrow 0} \frac{x \sin \sqrt{x}}{x^{3/2}} = \lim_{x \rightarrow 0} \frac{\sin \sqrt{x}}{\sqrt{x}} = 1$  by  $\lim_{u \rightarrow 0} \frac{\sin u}{u} = 1$  where  $u = \sqrt{x}$ . Letting  $M = 1$ ,  $x \sin \sqrt{x} = O(x^{3/2})$  as  $x \rightarrow 0$ .

(b)

For  $\int_0^\epsilon e^{-x^2} dx = O(\epsilon)$  as  $\epsilon \rightarrow 0$ , let  $f(x) = \int_0^\epsilon e^{-x^2} dx$  and  $g(x) = \epsilon$

$\lim_{\epsilon \rightarrow 0} \frac{\int_0^\epsilon e^{-x^2} dx}{\epsilon} = \lim_{\epsilon \rightarrow 0} \frac{\frac{d}{d\epsilon} \int_0^\epsilon e^{-x^2} dx}{\frac{d}{d\epsilon} \epsilon} = \lim_{\epsilon \rightarrow 0} \frac{e^{-\epsilon^2}}{1} = \lim_{\epsilon \rightarrow 0} e^{-\epsilon^2} = e^0 = 1$  Letting  $M = 1$  again,  $\int_0^\epsilon e^{-x^2} dx = O(\epsilon)$  as  $\epsilon \rightarrow 0$

2

$Ax = b$  where

$$A = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 + 10^{-10} & 1 - 10^{-10} \end{bmatrix}$$

$$b = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$x = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$A^{-1} = \begin{bmatrix} 1 - 10^{10} & 10^{10} \\ 1 + 10^{10} & -10^{10} \end{bmatrix}$$

$$\Delta b = \begin{bmatrix} \Delta b_1 \\ \Delta b_2 \end{bmatrix}$$

(a)

Find formula for the change in solution between  $x$  and  $\Delta x$

$A(x + \Delta x) = b + \Delta b$  gives  $Ax + A\Delta x = b + \Delta b$

Since  $Ax = b$ , this gives  $b + A\Delta x = b + \Delta b \rightarrow A\Delta x = \Delta b \rightarrow \Delta x = A^{-1}\Delta b$

$$\Delta x = A^{-1}\Delta b = \begin{bmatrix} 1 - 10^{10} & 10^{10} \\ 1 + 10^{10} & -10^{10} \end{bmatrix} \begin{bmatrix} \Delta b_1 \\ \Delta b_2 \end{bmatrix} = \begin{bmatrix} \Delta b_1 + 10^{10}(\Delta b_2 - \Delta b_1) \\ \Delta b_1 - 10^{10}(\Delta b_2 - \Delta b_1) \end{bmatrix}$$

(b)

Condition number of  $A$ .

$\kappa_2(A) = \|A\|_2 \|A^{-1}\|_2 = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)} = \sqrt{\frac{\lambda_{\max}(A^T A)}{\lambda_{\min}(A^T A)}}$  where  $\lambda$  represents the eigenvalues of  $A^T A$ . Letting  $\epsilon = 10^{-10}$  gives

$$A = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 + \epsilon & 1 - \epsilon \end{bmatrix} \text{ and } A^T = \frac{1}{2} \begin{bmatrix} 1 & 1 + \epsilon \\ 1 & 1 - \epsilon \end{bmatrix}$$

$$A^T A = \begin{bmatrix} \frac{1}{2} + \frac{1}{2}\epsilon + \frac{1}{4}\epsilon^2 & \frac{1}{2} - \frac{1}{4}\epsilon^2 \\ \frac{1}{2} - \frac{1}{4}\epsilon^2 & \frac{1}{2} - \frac{1}{2}\epsilon + \frac{1}{4}\epsilon^2 \end{bmatrix} \text{ gives } \lambda = \frac{1 + \frac{1}{2}\epsilon^2 \pm \sqrt{1 + \frac{1}{4}\epsilon^4}}{2}$$

$$\kappa_2(A) = \sqrt{\frac{1 + \frac{1}{2}\epsilon^2 + \sqrt{1 + \frac{1}{4}\epsilon^4}}{1 + \frac{1}{2}\epsilon^2 - \sqrt{1 + \frac{1}{4}\epsilon^4}}} \text{ gives}$$

$$\kappa_2(A) = \sqrt{\frac{1 + \frac{1}{2}(10^{-10})^2 + \sqrt{1 + \frac{1}{4}(10^{-10})^4}}{1 + \frac{1}{2}(10^{-10})^2 - \sqrt{1 + \frac{1}{4}(10^{-10})^4}}} = \sqrt{\frac{1 + \frac{1}{2}(10^{-20}) + \sqrt{1 + \frac{1}{4}(10^{-40})}}{1 + \frac{1}{2}(10^{-20}) - \sqrt{1 + \frac{1}{4}(10^{-40})}}} \approx 2 \times 10^{10}$$

(c) Let  $\Delta b_1$  and  $\Delta b_2$  be of magnitude  $10^{-5}$  and not necessarily the same value.

To measure relative error we want to look at the maximum absolute component of  $\Delta x$  via the infinity norm.

$$\Delta x = \begin{bmatrix} \Delta b_1 + 10^{10}(\Delta b_2 - \Delta b_1) \\ \Delta b_1 - 10^{10}(\Delta b_2 - \Delta b_1) \end{bmatrix} = \begin{bmatrix} 10^{-5} + 10^{10}(2 \times 10^{-5}) \\ 10^{-5} - 10^{10}(2 \times 10^{-5}) \end{bmatrix}$$

$$\text{Relative error (x)} = \frac{\|\Delta x\|_\infty}{\|x\|_\infty} = \frac{10^{-5} + 10^{10}(2 \times 10^{-5})}{1} = 10^{-5} + 10^{10}(2 \times 10^{-5}) = 200,000.00001$$

When the difference between  $\Delta b_1$  and  $\Delta b_2$  increases, the large condition number amplifies the relative error drastically as a result. Different values of perturbations are far more likely, which is a cause for concern on this problem.

### 3

$$f(x) = e^x - 1$$

(a)

The relative condition number  $\kappa(f(x)) = \left| \frac{xf'(x)}{f(x)} \right|$

$$f'(x) = e^x$$

$$\kappa(f(x)) = \left| \frac{xe^x}{e^x - 1} \right|$$

$$\text{For } x \rightarrow 0, \lim_{x \rightarrow 0} \left| \frac{xe^x}{e^x - 1} \right| \approx \left| \frac{xe^x}{x} \right| = 1$$

For  $x \rightarrow \infty, \lim_{x \rightarrow \infty} \left| \frac{xe^x}{e^x - 1} \right| \approx \left| \frac{xe^x}{e^x} \right| = |x|$  which grows linearly to infinity.  $f(x)$  is thus ill conditioned for very large  $x$ .

(b)

`y = math.e**x`

`return y-1`

This algorithm is not stable because as  $x$  decreases to very small values,  $\lim_{x \rightarrow 0} e^x$   $y$  becomes increasingly close to 1 and subtracting the like terms in step 2 leads to catastrophic cancellation.

(c)

$x = 9.999999995000000 \times 10^{-10}$  gives  $f(x) = 10^{-9}$  up to 16 decimal places.

```
In [ ]: x = 9.999999995000000e-10
y = math.e**x
print (y-1)
print(10e-10)
```

```
1.000000082740371e-09
1e-09
```

The algorithm only gives 7 correct decimal places. The decrease is expected due to the catastrophic cancellation mentioned when  $x$  is very small.

(d)

Taylor series for  $f(x) = e^x - 1 = x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$

We need Relative error =  $\left| \frac{\text{ExactValue} - \text{PolynomialApprox}}{\text{ExactValue}} \right| < 10^{-16}$

for  $n = 2$ , the first omitted term is  $\frac{x^3}{3!}$  which gives  $\text{RE} \approx \frac{\frac{x^3}{6}}{\frac{x}{6}} = \frac{x^2}{6}$  for  $x = 9.999999995000000e-10$ , this gives  $\text{RE} \approx 1.67^{-19}$

This gives  $f(x) \approx x + \frac{x^2}{2}$

(e)

```
In [17]: x = 9.999999995000000e-10
fx = x + ((x*x)/2)
fx
```

```
Out[17]: 1e-09
```

Which verifies  $f(x) \approx x + \frac{x^2}{2}$  for  $x = 9.999999995000000 \times 10^{-10}$  gives  $f(x) = 10^{-9}$  below the relative error.

### 4

$$2x - 1 = \sin x$$

(a)

Closed Interval  $[a, b]$  on which the equation has root  $r$  with IVT proof of  $r$ .

$$f(x) = 2x - 1 - \sin x$$

$$f(0) = 2(0) - 1 - \sin 0 = 0 - 1 - 0 = -1 \quad f(1) = 2(1) - 1 - \sin 1 = 2 - 1 - .841 = .159$$

Since  $f(0) < 0$  and  $f(1) > 0$ , there is a root  $r$  on the interval  $[0, 1]$ . Since  $f(x)$  is continuous and the function is bounded by  $f(0) = -1$  and  $f(1) = .159$ , there must be a root  $r$  such that  $f(r) = 0$ .

(b)

the value of the derivative  $f'(x) = 2 - \cos(x)$  will always stay above or equal to 1 for any  $x$  as  $\cos$  is bounded  $[-1, 1]$ , which means the function is strictly increasing for all  $x$ . A strictly increasing function can only cross  $y=0$  once, which we identified above.

(c)

```
In [1]: def bisection(a, b, func, tol):
        """
        f,a,b - function and endpoints of initial interval
        # tol - bisection stops when interval length < tol
        """
```

```
# first verify there is a root we can find in the interval

fa = func(a)
fb = func(b)

if (fa*fb>0):
    ier = 1
    astar = a
    return [astar, ier, "Not in interval"]

# verify end points are not a root
if (fa == 0):
    astar = a
    ier = 0
    return [astar, ier, "A is a root"]

if (fb == 0):
    astar = b
    ier = 0
    return [astar, ier, "B is a root"]

count = 0
d = .5 * (a + b)
while (abs(d-a)> tol):
    fd = func(d)
    if (fd == 0):
        astar = d
        ier = 0
        return [astar, ier, count]
    if (fa*fd<0):
        b = d
    else:
        a = d
        fa = fd
    d = .5 * (a+b)
    count = count +1
astar = d
ier = 0
return [astar, ier, count]
```

```
In [ ]: root, ier, iters = bisect(0,1,lambda x: 2*x - 1 - np.sin(x),1e-8)
```

```
if ier == 0:
    print("The root is:", root)
    print("The count is:", iters)
else:
    print("No root found")
```

The root is: 0.8878622129559517  
The count is: 26

5

(a)

Upper bound on iterations for bisection needed to approximate  $x^3 + x - 4 = 0$  lying in  $[1,4]$  with accuracy  $10^{-3}$ . Since  $f(x)$  is continuous over the interval and  $f(a)f(b) < 0$ , bisection generate a sequence of approximations converging to the root  $r$  such that  $|r - x_n| \leq \frac{b-a}{2^n}$ . For accuracy  $10^{-3}$ , set  $\frac{b-a}{2^n} \leq tol$  gives

$$\frac{4-1}{2^n} \leq 10^{-3} \rightarrow \frac{3}{2^n} \leq \frac{1}{1000} \rightarrow 2n \geq 3000 \rightarrow n \geq \log_2(3000) = \frac{\ln(3000)}{\ln(2)} \approx 11.55 \text{ upper bound for number of iterations.}$$

(b)

The code finds the root 1.380859375 in 8 iterations, which is under the upper bound determined above.

```
In [17]: root, ier, iters = bisect(1,4,lambda x: x**3 + x -4,1e-2)
```

```
if ier == 0:
    print("The root is:", root)
    print("The count is:", iters)
else:
    print("No root found")
```

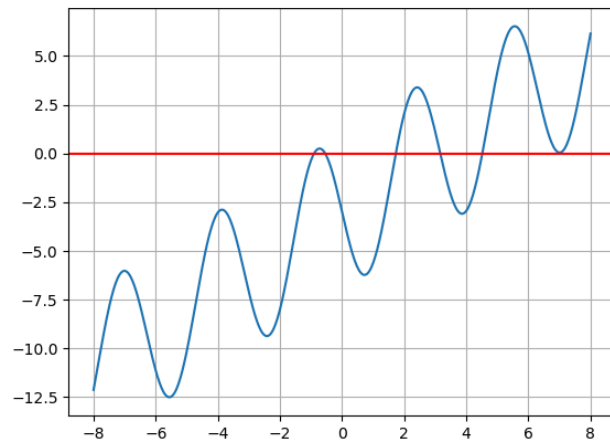
The root is: 1.380859375  
The count is: 8

6

(a)

```
In [ ]: x = np.linspace(-8, 8, 500)
y = x - 4 * np.sin(2 * x) - 3

plt.plot(x, y)
plt.axhline(0, color='red', linestyle='--')
plt.grid(True)
plt.show()
```



There are 5 crossings.

(b)

$$x_{n+1} = -\sin(2x_n) + \frac{5x_n}{4} - \frac{3}{4}$$

```
In [75]: def fixedpt(x0, f, tol, Nmax):
    ''' x0 = initial guess'''
    ''' Nmax = max number of iterations'''
    ''' tol = stopping tolerance'''

    count = 0
    v = []

    while (count < Nmax):
        count = count + 1
        x1 = f(x0)
        v.append(x1)
        if (abs(x1-x0) < tol):
            xstar = x1
            ier = 0
            vec_full = np.array(v)
            return [xstar, ier, vec_full]
        vec_full = np.array(v)
        x0 = x1
        xstar = x1
        ier = 1
    return [xstar, ier, vec_full]

In [76]: root1, ier, vector = fixedpt(-1, lambda x: -np.sin(2*x) + (5*x)/4 - 3/4, 1e-9, 100)

if ier == 0:
    print("The root is:", root1)
else:
    print("No root found")

No root found

In [77]: root2, ier, vector = fixedpt(0, lambda x: -np.sin(2*x) + (5*x)/4 - 3/4, 1e-9, 100)

if ier == 0:
    print("The root is:", root2)
else:
    print("No root found")

The root is: -0.5444424009452844

In [78]: root3, ier, vector = fixedpt(1.8, lambda x: -np.sin(2*x) + (5*x)/4 - 3/4, 1e-9, 100)

if ier == 0:
    print("The root is:", root3)
else:
    print("No root found")

The root is: 3.1618264861802765

In [79]: root4, ier, vector = fixedpt(5, lambda x: -np.sin(2*x) + (5*x)/4 - 3/4, 1e-9, 100)

if ier == 0:
    print("The root is:", root4)
else:
    print("No root found")

No root found
```

The fixed point method is only able to find 2 of the 5 roots due to the requirement that the absolute value of the derivative must be less than 1 at the root for convergence. For 3 of the roots, the derivative is greater than 1.